

Close

B S CHENTHIL HARI
Exercise 2: Euclidean Distance Calculation

B S CHENTHIL HARI

Exercise 2: Euclidean Distance Calculation

Calculate the distance between a new entry and other existing values using the Euclidean distance formula.

```
import pandas as pd
import numpy as np
```

▼ Create Dataset

```
data = pd.DataFrame({
    'Brightness': [40, 50, 60, 10, 70, 60, 25],
    'Saturation': [20, 50, 90, 25, 70, 10, 80],
    'Class': ['Red', 'Blue', 'Blue', 'Red', 'Blue', 'Red', 'Blue']
})
print(data)
```

	Brightness	Saturation	Class
0	40	20	Red
1	50	50	Blue
2	60	90	Blue
3	10	25	Red
4	70	70	Blue
5	60	10	Red
6	25	80	Blue

▼ Define Euclidean Distance Function

```
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))
```

▼ Calculate Distance from New Entry to All Existing Values

```
new_entry = np.array([45, 35]) # Brightness=45, Saturation=35

distances = []
for idx, row in data.iterrows():
    existing_point = np.array([row['Brightness'], row['Saturation']])
    dist = euclidean_distance(new_entry, existing_point)
    distances.append(dist)

data['Distance from (45,35)'] = [round(d, 2) for d in distances]
print(data.to_string(index=False))
```

Brightness	Saturation	Class	Distance from (45,35)
40	20	Red	15.81
50	50	Blue	15.81
60	90	Blue	57.01
10	25	Red	36.40
70	70	Blue	43.01
60	10	Red	29.15
25	80	Blue	49.24

▼ Classify by Nearest Neighbor (k=1)

```
min_idx = data['Distance from (45,35)'].idxmin()
print(f"Nearest neighbor is index {min_idx}")
print(f" Brightness={data.loc[min_idx, 'Brightness']}, Saturation={data.loc[min_idx, 'Saturation']}")
print(f" Distance = {data.loc[min_idx, 'Distance from (45,35)']}")
print(f" Class = {data.loc[min_idx, 'Class']}")
print(f"\nPrediction: The new entry (45, 35) is classified as '{data.loc[min_idx, 'Class']}'")
```

```
Nearest neighbor is index 0  
Brightness=40, Saturation=20  
Distance = 15.81  
Class = Red
```

Prediction: The new entry (45, 35) is classified as 'Red'